

Bocconi

COMPUTER SCIENCE

code 30424 a.y. 2024-2025

Lesson 22

Essentials on algorithms



Universit 
Bocconi
MILANO

Objectives of the lesson

- Understand the essentials on algorithms
- Understand algorithm complexity
- Understand algorithm efficiency

What is an Algorithm?

- An algorithm is a **well-defined procedure** that takes inputs and produces outputs in a **finite number of steps**
- Algorithms are used to define the solution to problems of various kinds, including simple calculations such as:

$$\textit{hypotenuse} = \sqrt{a^2 + b^2}$$

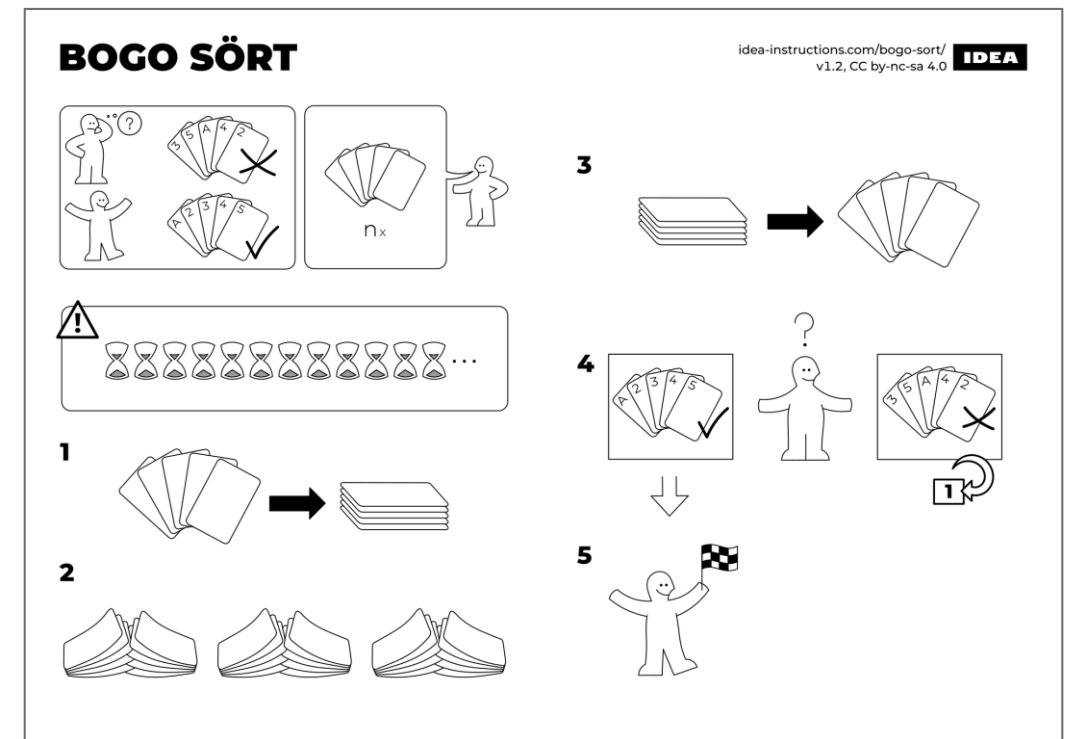
- step 1:** calculate the square of the input **a**
- step 2:** calculate the square of the input **b**
- step 3:** add the results of **step 1** and **step 2**
- step 4:** extract the square root of the result of **step 3**



Everyday algorithm examples

Any finite sequence of well-defined instructions for solving a class of problems can be defined an algorithm:

- ✓ Cake recipe
- ✓ IKEA assembly directions
- ✓ Bedtime routine
- ✓ Home to University route
- ✓ SERP ranking



Bogo sort, also known as stupid sort, is a simple, but highly inefficient sorting algorithm, which simply shuffles the elements until they are sorted.

Algorithm representation methods

There are different ways to express or communicate the steps of an algorithm:

- ✓ **Natural Language**
using regular spoken or written language (like instructions in a manual)
- ✓ **Flowcharts**
using diagrams with shapes and arrows to show steps visually
- ✓ **Pseudocode**
writing steps in a way similar to programming languages, but simpler and easier for humans to read and understand

Algorithm main features

To ensure that an algorithm can be followed reliably to solve a particular problem it must fulfil the following requirements:

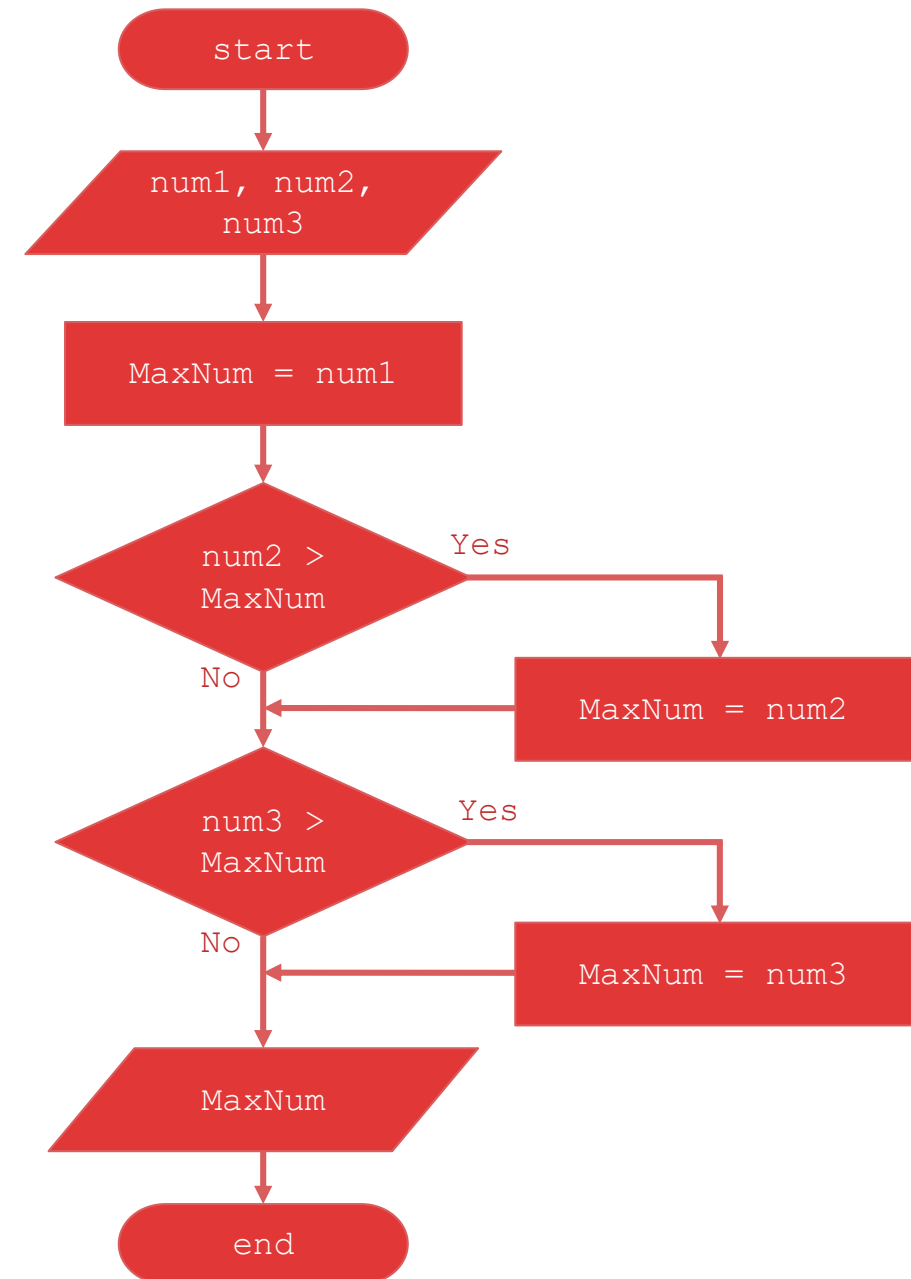
- ✓ **Finiteness:** the algorithm must always terminate after a finite number of steps
- ✓ **Clarity:** every step of the algorithm must be clear and unambiguous
- ✓ **Defined inputs:** inputs needed to solve the problem must be well-defined
- ✓ **Defined outputs:** the algorithm must produce at least one output
- ✓ **Effectiveness:** all the steps must be carried out in a reasonable amount of time, using simple, well-defined operations

Exercise:

Given three numbers (num1, num2 and num3) define the algorithm to return the highest value, using only the comparison operators.

Solution:

```
def maximum(num1, num2, num3):  
    MaxNum = num1  
  
    if num2 > MaxNum:  
        MaxNum = num2  
  
    if num3 > MaxNum:  
        MaxNum = num3  
  
    return MaxNum
```



A classic algorithm

A classic example of an algorithm is the one used to **define the pairings** of elements from two different groups, based on the highest **degree of affinity**

The most common scenarios where this algorithm is used are:

- ✓ College admissions (students and universities)
- ✓ Job placements (companies and candidates)
- ✓ Organ transplants (patients and donors)
- ✓ Online dating platforms



The Gale-Shapley algorithm

- Gale-Shapley algorithm is used to find a **stable matching** between two equally sized sets of elements, given a list of preferences for each element
- Introduced by mathematicians David Gale and Lloyd Shapley in 1962, the goal was to solve the “stable marriage problem”
- In 1984 Alvin E. Roth pointed out its prior application (since the 50s) as the “Boston Pool algorithm” used by the National Resident Matching Program
- The algorithm started as a mathematical curiosity but became so influential in economics and everyday life that it earned **Lloyd Shapley** and **Alvin E. Roth** a **Nobel Prize in Economics** in 2012 (David Gale had died in 2008)



Lloyd Shapley and Alvin E. Roth

Gale-Shapley theorem

- For an equal number of participants in **two groups** (such as universities and students), each with a comprehensive and rigorous **order of preference** for the members of the other group, there always exists at least one **stable matching**
- A **stable matching** is defined as a set of matches in which there is no pair of participants (one from each group) who would both prefer to be matched with each other over their current matches
- In other words, no two people can form a pair that would make both more satisfied, preventing therefore any “instability” in the matching

Gale-Shapley: a practical application

- Consider the case of a set of **n** different **universities** that offers an equal number of **n** different **students** the opportunity to enroll in their courses
- Each university and each student expresses their preferences, **ranking** the elements of the other group
- Giving the **priority of choice** to universities, the algorithm iteratively proposes matches until a **stable configuration** is found

Universities-students matching: the preferences

 preferences	1 st	2 nd	3 rd	4 th
Bocconi	Mario	Alex	John	Sophie
Hec	John	Sophie	Mario	Alex
Esade	John	Mario	Sophie	Alex
LSE	Alex	Sophie	John	Mario

Assumption:

- **Universities** propose admission to their courses based on their order of preference for students
- **Students** accept the proposal from the university that they have ranked highest

 preferences	1 st	2 nd	3 rd	4 th
Alex	Esade	LSE	Hec	Bocconi
Mario	Esade	Hec	Bocconi	LSE
John	Bocconi	Hec	Esade	LSE
Sophie	LSE	Esade	Hec	Bocconi



Universities-students matching

 preferences	1 st	2 nd	3 rd	4 th
Bocconi	Mario	Alex	John	Sophie
Hec	John	Sophie	Mario	Alex
Esade	John	Mario	Sophie	Alex
LSE	Alex	Sophie	John	Mario

- 1) to start, **Bocconi** proposes to the student it prefers the most, which is **Mario**
- 2) since Mario is currently unmatched, he accepts the proposal

3) now Bocconi and Mario are matched

 preferences	1 st	2 nd	3 rd	4 th
Alex	Esade	LSE	Hec	Bocconi
Mario	Esade	Hec	Bocconi	LSE
John	Bocconi	Hec	Esade	LSE
Sophie	LSE	Esade	Hec	Bocconi



Universities-students matching

 preferences	1 st	2 nd	3 rd	4 th
Bocconi	Mario	Alex	John	Sophie
Hec	John	Sophie	Mario	Alex
Esade	John	Mario	Sophie	Alex
LSE	Alex	Sophie	John	Mario

- 4) now **Hec** proposes to the student it prefers the most, which is **John**
- 5) since John is currently unmatched, he accepts the proposal

6) now Hec and John are matched

 preferences	1 st	2 nd	3 rd	4 th
Alex	Esade	LSE	Hec	Bocconi
Mario	Esade	Hec	Bocconi	LSE
John	Bocconi	Hec	Esade	LSE
Sophie	LSE	Esade	Hec	Bocconi



Universities-students matching

 preferences	1 st	2 nd	3 rd	4 th
Bocconi	Mario	Alex	John	Sophie
Hec	John	Sophie	Mario	Alex
Esade	<i>John</i>	Mario	Sophie	Alex
LSE	Alex	Sophie	John	Mario

- 7) now **Esade** proposes to **John**, but John is already matched with Hec
- 8) so the algorithm checks whether John prefers Esade proposal to his current match with Hec
- 9) in this case John prefers Hec

10) so John rejects Esade and **Esade remains unmatched**

 preferences	1 st	2 nd	3 rd	4 th
Alex	Esade	LSE	Hec	Bocconi
Mario	Esade	Hec	Bocconi	LSE
John	Bocconi	Hec	<i>Esade</i>	LSE
Sophie	LSE	Esade	Hec	Bocconi



Universities-students matching

 preferences	1 st	2 nd	3 rd	4 th
Bocconi	<i>Mario</i>	Alex	John	Sophie
Hec	John	Sophie	Mario	Alex
Esade	<i>John</i>	Mario	Sophie	Alex
LSE	Alex	Sophie	John	Mario

- 11) next **Esade** proposes to **Mario**, but Mario is already matched with Bocconi
- 12) so the algorithm checks whether Mario prefers Esade proposal to his current match with Bocconi
- 13) in this case Mario prefers Esade

- 14) so now Esade and Mario are matched
- 15) but at this point Bocconi becomes unmatched

 preferences	1 st	2 nd	3 rd	4 th
Alex	Esade	LSE	Hec	Bocconi
Mario	Esade	Hec	<i>Bocconi</i>	LSE
John	Bocconi	Hec	<i>Esade</i>	LSE
Sophie	LSE	Esade	Hec	Bocconi



Universities-students matching

 preferences	1 st	2 nd	3 rd	4 th
Bocconi	<i>Mario</i>	Alex	John	Sophie
Hec	John	Sophie	Mario	Alex
Esade	<i>John</i>	Mario	Sophie	Alex
LSE	Alex	Sophie	John	Mario

16) so now **Bocconi** proposes to **Alex**

17) since Alex is currently unmatched, she accepts the proposal

18) so now Bocconi and Alex are matched

 preferences	1 st	2 nd	3 rd	4 th
Alex	Esade	LSE	Hec	Bocconi
Mario	Esade	Hec	<i>Bocconi</i>	LSE
John	Bocconi	Hec	<i>Esade</i>	LSE
Sophie	LSE	Esade	Hec	Bocconi



Universities-students matching

 preferences	1st	2nd	3rd	4th
Bocconi	Mario	Alex	John	Sophie
Hec	John	Sophie	Mario	Alex
Esade	John	Mario	Sophie	Alex
LSE	Alex	Sophie	John	Mario

- 19) now also **LSE** proposes to **Alex**, who is already matched with Bocconi
- 20) so the algorithm checks whether Alex prefers LSE proposal to her current match with Bocconi
- 21) in this case Alex prefers LSE

- 22) so now LSE and Alex are matched
- 23) but at this point Bocconi becomes again unmatched

 preferences	1st	2nd	3rd	4th
Alex	Esade	LSE	Hec	<i>Bocconi</i>
Mario	Esade	Hec	<i>Bocconi</i>	LSE
John	Bocconi	Hec	<i>Esade</i>	LSE
Sophie	LSE	Esade	Hec	Bocconi



Universities-students matching

 preferences	1st	2nd	3rd	4th
Bocconi	Mario	Alex	John	Sophie
Hec	John	Sophie	Mario	Alex
Esade	John	Mario	Sophie	Alex
LSE	Alex	Sophie	John	Mario

24) now **Bocconi** proposes to **John**

25) so the algorithm checks whether John prefers Bocconi proposal to his current match with Hec

26) in this case John prefers Bocconi

27) so now Bocconi and John are matched

28) but at this point Hec becomes unmatched

 preferences	1st	2nd	3rd	4th
Alex	Esade	LSE	Hec	<i>Bocconi</i>
Mario	Esade	Hec	<i>Bocconi</i>	LSE
John	Bocconi	<i>Hec</i>	<i>Esade</i>	LSE
Sophie	LSE	Esade	Hec	Bocconi



Universities-students matching

 UNIVERSITY	preferences			
	1 st	2 nd	3 rd	4 th
Bocconi	Mario	Alex	John	Sophie
Hec	John	Sophie	Mario	Alex
Esade	John	Mario	Sophie	Alex
LSE	Alex	Sophie	John	Mario

29) so now **Hec** proposes to **Sophie**

30) since Sophie is currently unmatched, she accepts the proposal

31) now also Hec and Sophie are matched

 STUDENT	preferences			
	1 st	2 nd	3 rd	4 th
Alex	Esade	LSE	Hec	Bocconi
Mario	Esade	Hec	Bocconi	LSE
John	Bocconi	Hec	Esade	LSE
Sophie	LSE	Esade	Hec	Bocconi

Universities-students stable matches

	
Bocconi	John
Hec	Sophie
Esade	Mario
LSE	Alex

Observations:

- The **sequence** of the universities is not relevant: we can arbitrarily choose which of the unmatched university to pair next and we will obtain the same stable matching
- The algorithm always finds a stable matching, but the result is different according to who plays the role of the **proposer**
- The approach adopted is **university-optimal** and **student-pessimal**, that is, in a **stable matching**:
 - each university gets the **best** possible student it could have
 - each student gets the **least** favorable university



Complexity in Gale-Shapley algorithm

- In our example, with two groups of 4 participants (i.e. $n = 4$) it took 8 steps to achieve a stable matching
- In an ideal scenario, each university would have a different preferred student, making the needed number of steps = n
- In the worst (**theoretical**) case, each proposer could be rejected by every member except their last choice, making the needed number of steps = n^2
- In the worst realistic scenario, the maximum number of steps needed to achieve a stable matching is $(n^2 + n)/2$

Current example

 UNIVERSITY	1st	2nd	3rd	4th
Bocconi	Mario	Alex	John	Sophie
Hec	John	Sophie	Mario	Alex
Esade	John	Mario	Sophie	Alex
LSE	Alex	Sophie	John	Mario

Ideal scenario

 UNIVERSITY	1st	2nd	3rd	4th
Bocconi	Alex	Mario	John	Sophie
Hec	Mario	John	Sophie	Alex
Esade	John	Sophie	Alex	Mario
LSE	Sophie	Alex	Mario	John

Worst realistic scenario

 UNIVERSITY	1st	2nd	3rd	4th
Bocconi	Alex	Mario	John	Sophie
Hec	Alex	Mario	John	Sophie
Esade	Alex	Mario	John	Sophie
LSE	Alex	Mario	John	Sophie

	1st	2nd	3rd	4th
Alex	LSE	Esade	Hec	Bocconi
Mario	LSE	Esade	Hec	Bocconi
John	LSE	Esade	Hec	Bocconi
Sophie	LSE	Esade	Hec	Bocconi

Algorithm complexity and efficiency

Algorithm **efficiency** refers to how well an algorithm uses computational resources to solve a problem: an efficient algorithm completes its task using minimal resources

Complexity is a formal way to describe the efficiency of an algorithm, both in terms of:

- ✓ **Time complexity:**
how many elementary steps are taken as the input size grows
- ✓ **Space complexity:**
how memory usage increases as the input size grows

Big-O notation

When talking about time/space complexity it is convenient to ignore details.
Let n be the size of the input:

1. There is **some difference** between an algorithm that, for example, runs in time $3n + 17$ and one that runs in time $2n + 10\sqrt{n} + 100$
 $\Rightarrow$ if $n = 1,000$: the first takes 3,017 steps while the second 2,417
2. There is **a lot of difference** between an algorithm that, for example, runs in time $5n$ and one that runs in time $0.5n^3$
 $\Rightarrow$ if $n = 1,000$: the first takes 5,000 steps the second takes 500,000,000 steps!

Big-O notation

The **Big-O** notation is used to conveniently ignore the details and focus on the **dominating** terms.

For example:

we say an algorithm runs in time $O(n)$ if it takes approximately n steps

Thus, considering the two cases of the previous slide:

- ✓ in case **1.** both algorithm run in time $O(n)$
- ✓ in case **2.** the first algorithm runs in time $O(n)$ but the second runs in time $O(n^3)$

Big-O notation

Big-O notation is the standard way to express algorithm complexity by describing the **upper** limit of an algorithm's growth rate

In other words, **Big-O** notation express **how fast** the requirement of resources **increases** as the size of the input (n) grows:

- ✓ $O(1)$ Constant (the requirement of resources is not affected by the size of the input)
- ✓ $O(\log_2 n)$ Logarithmic increase (the use of resources grows slowly)
- ✓ $O(n)$ Linear increase (the use of resources grows proportionally to the input)
- ✓ $O(n^2)$ Quadratic increase (the use of resources grows faster than the input)

We can therefore say that the Gale-Shapley algorithm can (rarely) be as fast as $O(n)$ and in the worst case it runs in time $O(n^2)$

Sorting algorithms

- The impact of **efficiency** is particularly evident observing different **sorting algorithms** used to arrange elements of a list or collection
- The **elements** to sort can be numbers, characters, strings, or complex structures
- Sorting is performed – in either ascending or descending order – based on a **key** or **property** of the elements (commonly the value)
- The sorting process involves comparing these keys to determine the correct sequence

Time and space complexity

Based on how memory resources are used, sorting algorithms can be classified as:

- **In-place** algorithms
they sort the data without needing extra storage, modifying the original collection
- **Out-of-place** algorithms
they require additional memory to complete the sorting process, often creating copies or auxiliary data structures

Thus, the efficiency of sorting algorithms is based on how quickly they run (**time complexity**) and how much additional memory they use (**space complexity**)

Most common sorting algorithms

The most common sorting algorithms used in computer science and practical applications include:

- ✓ Bubble sort
- ✓ Selection sort
- ✓ Insertion sort
- ✓ Merge sort

Bubble sort

It is a simple comparison-based algorithm that works by:

- ✓ repeatedly stepping through the list
- ✓ comparing adjacent pairs of elements
- ✓ swapping them if they are in the wrong order

The process repeats until the list is sorted

- **Pros:** it is very simple, easy to implement and does not require extra memory (in-place)
- **Cons:** it is very inefficient for large datasets



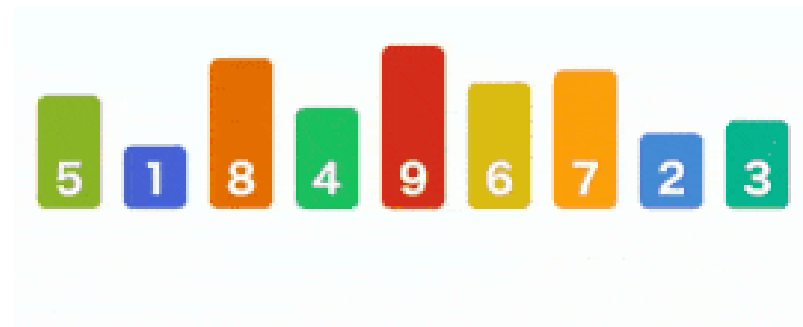
Selection sort

It is another simple comparison-based algorithm that works by:

- ✓ dividing the list into a sorted and an unsorted part
- ✓ repeatedly finding the minimum element from the unsorted part
- ✓ moving it to the end of the sorted part

The process repeats until the list is sorted

- **Pros:** simple to understand and implement
- **Cons:** time complexity (typically makes fewer swaps than bubble sort but more comparisons)



Insertion sort

It builds the sorted array one item at a time, by:

- ✓ removing each element from the input data
- ✓ finding its correct position in the sorted list and inserting it there
- ✓ shifting the other elements as needed

The process repeats until the list is sorted

- **Pros:** efficient for small or nearly sorted datasets
- **Cons:** time complexity in case of large and/or unsorted datasets



Merge sort

Merge sort is a classic “divide and conquer” algorithm that works by:

- ✓ dividing the list into subsequent halves
- ✓ recursively sorting each half
- ✓ merging the sorted halves into a sorted list

The process ends when no unsorted halves are left

- **Pros:** efficient also for very large datasets
- **Cons:** more complex to implement and requires extra memory for merging



Summary Table

Algorithm	Time complexity		Space complexity	
	Best case	Worst case	Memory usage	Complexity
Bubble sort	$O(n)$	$O(n^2)$	In-place	$O(1)$
Selection sort	$O(n^2)$	$O(n^2)$	In-place	$O(1)$
Insertion sort	$O(n)$	$O(n^2)$	In-place	$O(1)$
Merge sort	$O(n \log n)$	$O(n \log n)$	Out-of-place	$O(n)$



The most effective choice

Although the summary table clearly shows the differences in complexity between the four approaches, **no single sorting** algorithm is universally the **best**

The most suitable choice depends on many factors that vary with the context in which the sorting is being performed:

- ✓ Nature of the **input data** (size of the dataset, order of elements, amount of duplicates, ...)
- ✓ **Resource** constraints (memory availability, time constraints)
- ✓ Trade-off between **implementation** simplicity and **performance**
- ✓ Hardware and system **architecture**

In practice

The video shows the differences between various sorting algorithms under various condition:

- random starting order
- presence of duplicates
- reverse starting order
- partial starting order

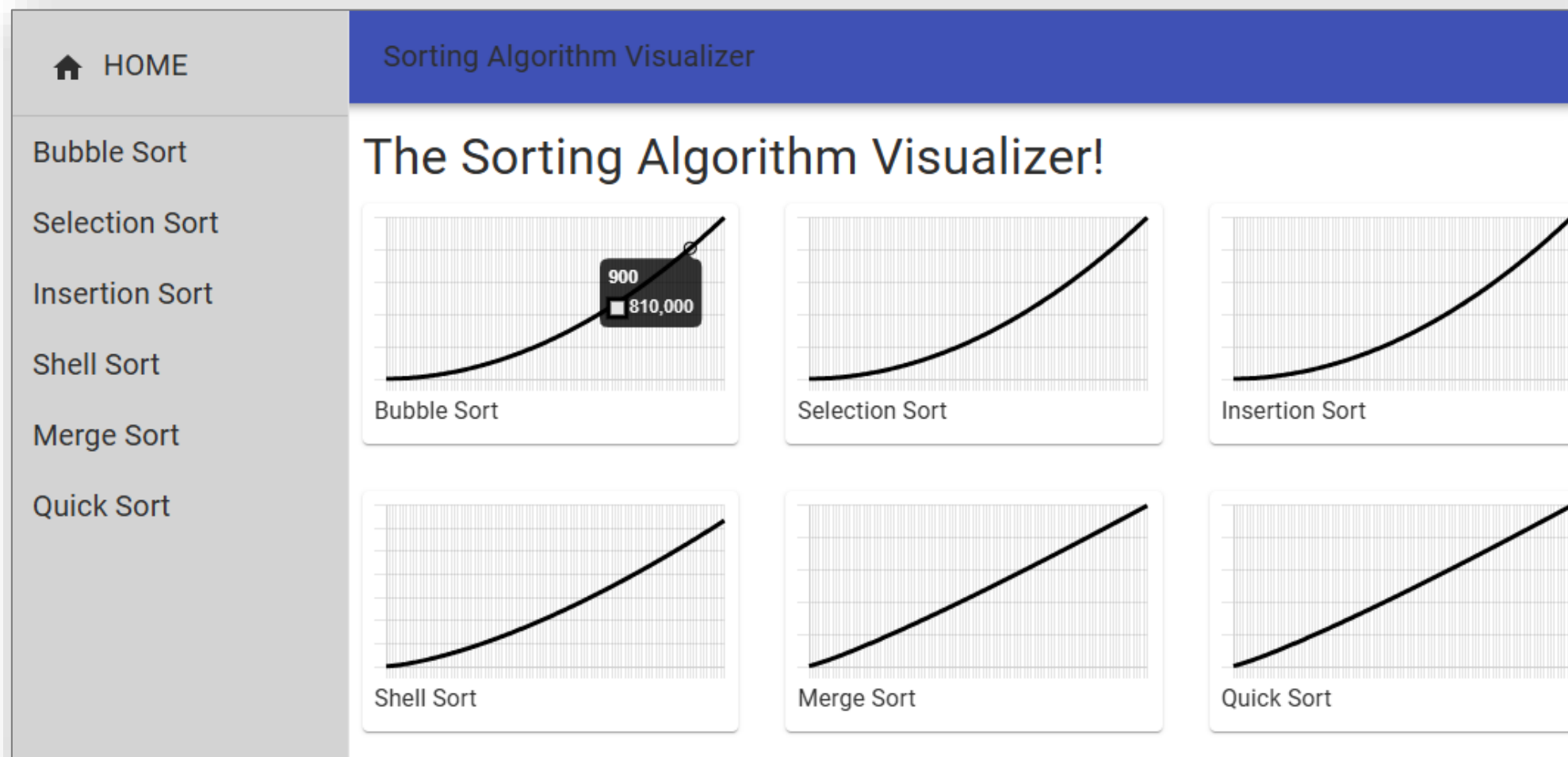
<https://www.youtube.com/watch?v=ZZuD6iUe3Pc>



Random



To further explore the observations



Assignment (optional)

1. Take a deck of playing cards
2. Shuffle them thoroughly
3. Sort them using one of the algorithms just analyzed
4. Repeat steps 2 and 3 for each of the algorithms just analyzed

For each of the algorithms, carefully observe:

- The time needed to achieve the final result
- The area occupied on your desk



Topics to learn with today's lecture

- Gale-Shapley theorem application
- Algorithmic approach to sorting

Study materials

This **set of slides** has to be considered the study material for the contents of **Lesson 22**

Any links to external resources have to be considered as supplementary material for further exploration of the topics covered